

BAN404 Exam Overview

BAN404 — Eksamensoversikt 2026

Eksamensdato: 19. mai 2026 | Varighet: 6 timer | Totalt: 100 poeng

To nivåer av signaler:

1. Historiske mønstre (fra 2024- og 2025-eksamen): nyttig kontekst, men ingen garanti. En metode som ikke har dukket opp er ikke trygt å utelukke, og en som har dukket opp er ikke garantert å komme igjen.

2. Datasignalet (fra utdelte filer): det sterkeste signalet. $n=5000$ i `task1_data.csv` gjør LOOCV praktisk umulig — K-fold er naturlig. Ikke-lineære mønstre i x_2 og x_3 peker mot GAM. Binær klassifikasjon med 14.6% churn peker mot standard Task 2-rammeverk. Datasettene deles ut dagen i forveien nettopp for at man kan tilpasse forberedelsene.

Mål: Ingen metode fra pensum utelukkes — men datasignalets vurdering er ærlig markert. Kode i trykte dokumenter refereres. Ny kode skrives ut med inline #-kommentarer.

Hva vi vet med sikkerhet om eksamensstrukturen

Basert på 2024- og 2025-eksamen er strukturen stabil, men innholdet varierer:

Deloppgave	Hva den alltid tester	2024	2025
1(a)	Kodelesing av en metode (f og/eller g)	KNN lokal reg.	Ridge reg.
1(b)	Anvende metoden på data	LOOCV for opt. K	OLS vs Ridge ulike λ
1(c)	Kryssvalidering	Plot med opt. K	LOOCV for opt. λ
1(d)	Bootstrap	Multivariat utvidelse	Bootstrap Var(y)
1(e)	Utvidelse / ikke-linearitet	Backfitting	GAM
2(a–e)	Anvendt analyse med datasett	Airline satisfaction	Insurance claims

Legg merke til at **2024 og 2025 hadde svært ulik struktur** innenfor det samme rammeverket. Ikke anta at 2026 følger 2025-malen.

TASK 1 — Metodikk (50 poeng)

1(a) — Kodelesing (10 poeng)

En eller to funksjoner presenteres. Du skal forklare hva de gjør, hva de merkede linjene gjør, og hva som skjer når parameteren endres.

Spørsmålene er alltid (tilnærmet) identiske: - **i.** Forklar hva `f` og `g` gjør steg for steg. Hvilken kursmetode er dette? - **ii.** Forklar linjene merket `(*)`, `(**)`, `(***)`. - **iii.** Hva skjer med koeffisientene/prediksjonene når parameteren (λ , K , osv.) øker? Hva er den viktigste forskjellen fra [annen metode]?

Alle metoder fra pensum som kan dukke opp i 1(a):

Lasso-regresjon

Kode: `practice_exam_A`, `Predicted-26Exam-SOLVED`

```
def f(b, X, y, la):
    rss      = np.sum((y - X @ b) ** 2)
    penalty  = la * np.sum(np.abs(b))    # (*) L1-straff: absoluttverdier → eksakt null
                                           mulig
    return rss + penalty

def g(X, y, la):
    p        = X.shape[1]
    b0       = np.mean(y)                # (**) intercept = mean(y), aldri straffet
    y_c      = y - b0
    X_c      = X - X.mean(axis=0)        # (***) demean X (ikke standardiser!)
    b_start  = np.zeros(p)
    result   = minimize(f, b_start, args=(X_c, y_c, la), method='L-BFGS-B')
    return b0, result.x

# Når λ↑: koeffisienter krymper → noen blir eksakt null (variabelutvalg)
# Skiller seg fra Ridge: Ridge bruker b**2 og kan ALDRI nulle koeffisienter
```

Ridge-regresjon

Dukket opp i 2025-eksamen (R-kode). Python-versjon — ikke i noe trykt dokument:

```
def f_ridge(b, X, y, la):
    rss = np.sum((y - X @ b) ** 2)
    penalty = la * np.sum(b ** 2)
    # (*) L2-straff: kvadrerte koeff → aldri eksakt null
    return rss + penalty

def g_ridge(X, y, la):
    p = X.shape[1]
    b0 = np.mean(y) # (**) intercept = mean(y), aldri straffet
    y_c = y - b0
    X_c = X - X.mean(axis=0) # (***) demean X (ikke standardiser!)
    b_start = np.zeros(p)
    result = minimize(f_ridge, b_start, args=(X_c, y_c, la), method='L-BFGS-B')
    return b0, result.x

# Når λ↑: alle koeffisienter krymper mot null, men INGEN blir eksakt null
# Skiller seg fra Lasso: Lasso bruker abs(b) og kan gi eksakt null (variabelutvalg)
```

KNN lokal regresjon

Dukket opp i 2024-eksamen. Kode: `practice_exam_B`

Nøkkellinjer: `d = np.abs(x - x0)` ← avstand, `o = np.argsort(d)[:K]` ← K nærmeste,
`LinearRegression().fit(x[o], y[o])` ← lokal lineær modell på de K nærmeste.

Når $K \uparrow$: glattere kurve, høyere bias, lavere varians. $K=1$ = overfitting; $K=n$ = global OLS.

K-fold kryssvalidering (som hoved-1a-metode)

Dukket opp i `BAN404_mock_exam-2` og `TestExam1`. Ikke i faktisk eksamen ennå — men mulig.

```
def kfold_cv(X, y, la, K=5, seed=42):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(y)) # (*) bland indekser tilfeldig
    folds = np.array_split(idx, K) # (**) del i K like store grupper
    mse_k = np.zeros(K)
    for k in range(K):
        te = folds[k]
        tr = np.concatenate([folds[j] for j in range(K) if j != k])
        b0k, b1k = g(X[tr], y[tr], la)
        X_c_te = X[te] - X[tr].mean(axis=0) # (***) bruk treningsfoldets gjennomsnitt!
        mse_k[k] = np.mean((y[te] - (b0k + X_c_te @ b1k)) ** 2)
    return np.mean(mse_k)

# (**) er kritisk: bruk treningsfoldets mean, ikke hele datasettets mean
# → ellers lekker testinformasjon inn i feature-konstruksjonen
```

Polynomial regresjon med K-fold

Dukket opp i `TestExam1` og `BAN404_mock_exam-2`.

```

def cv_poly(x, y, K=5, degree=3, seed=42):
    from sklearn.preprocessing import PolynomialFeatures
    from sklearn.linear_model import LinearRegression
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(y))
    folds = np.array_split(idx, K)
    mse_k = np.zeros(K)
    for k in range(K):
        te, tr = folds[k], np.concatenate([folds[j] for j in range(K) if j != k])
        poly = PolynomialFeatures(degree=degree)
        X_tr = poly.fit_transform(x[tr].reshape(-1,1)) # (*) tilpass PF på
        # treningsfold
        X_te = poly.transform(x[te].reshape(-1,1)) # (**) transformer test med
        # same PF
        m = LinearRegression().fit(X_tr, y[tr])
        mse_k[k] = np.mean((y[te] - m.predict(X_te)) ** 2) # (***) test-MSE
    return np.mean(mse_k)

```

Bagging (Bootstrap Aggregating)

Dukket opp i [BAN404_mock_exam-2](#) (1e). Mulig som kodelesing.

```

def bagging(X_tr, y_tr, X_te, B=200, seed=42):
    rng = np.random.default_rng(seed)
    n = len(y_tr)
    preds = np.zeros((len(X_te), B))
    for b in range(B):
        idx = rng.choice(n, size=n, replace=True) # (*) bootstrap-utvalg med
        # tilb.l.
        X_b, y_b = X_tr[idx], y_tr[idx] # (**) bootstrap-datasett
        from sklearn.tree import DecisionTreeRegressor
        tree = DecisionTreeRegressor().fit(X_b, y_b)
        preds[:,b] = tree.predict(X_te) # (***) prediksjon fra tre b
    return preds.mean(axis=1) # gjennomsnitt over B trær = bagging-prediksjon
# Bagging reduserer varians ved å gjennomsnitte mange trær trent på litt ulike data

```

LDA / QDA / Logistisk regresjon (sammenligning)

Dukket opp i [BAN404_mock_exam-3](#) (1e). Mulig som kodelesing eller konseptuell oppgave.

```

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

lda = LinearDiscriminantAnalysis()
qda = QuadraticDiscriminantAnalysis()
lr = LogisticRegression()

for name, model in [("LDA", lda), ("QDA", qda), ("LR", lr)]:
    cv_acc = cross_val_score(model, X, y, cv=5, scoring='accuracy') # (*) 5-fold CV
    print(f"{name}: CV accuracy = {cv_acc.mean():.4f} ± {cv_acc.std():.4f}")

# LDA antar felles kovariansmatrise for alle klasser → lineær grense
# QDA tillater ulike kovariansmatriser → kvadratisk grense, mer fleksibel
# LR: ingen distribusjonsantagelse, modellerer  $P(Y=1|X)$  direkte

```

PCA (Principal Component Analysis)

Dukket opp i [BAN404_mock_exam-3](#) (1c). I pensum (Forelesning 14).

```

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

scaler = StandardScaler()
X_std = scaler.fit_transform(X) # (*) standardiser – PCA er skala-sensitiv

pca = PCA()
pca.fit(X_std)

# Andel forklart varians per komponent
var_exp = pca.explained_variance_ratio_ # (**) eigenvalues / total varians
cum_var = np.cumsum(var_exp) # (***) kumulativ varians

# Scree plot
plt.plot(range(1, len(var_exp)+1), var_exp, 'b-o')
plt.xlabel("Principal Component"); plt.ylabel("Proportion of Variance Explained")

# Antall komponenter for 90% forklart varians
n_comp = np.argmax(cum_var >= 0.90) + 1
print(f"Komponenter for 90% varians: {n_comp}")
print("Loadings PC1:", pca.components_[0]) # koeffisienter for første PC

```

1(b) — Anvende metoden (10 poeng)

Tilpass med OLS + to parameterverdier. Presenter i tabell. Kommenter forskjeller.

Kode: Se [practice_exam_A](#) (Lasso) eller [practice_exam_B](#) (KNN).

Uansett hvilken metode — svar alltid på: - Hva skjer med parameterverdiene (koeffisienter, K, etc.) når parameteren endres? - Hvilken parameterverdi gir tilsynelatende best tilpasning visuelt / i tabellen? - Utfører noen av de regulariserte modellene variabelutvalg?

1(c) — Kryssvalidering (10 poeng)

Datasignalet er tydelig her: `task1_data.csv` har $n=5000$ observasjoner. LOOCV med $n=5000$ over 40 λ -verdier = 200 000 modell-tilpasninger — ikke gjennomførbart på 6 timer. **K-fold CV er det naturlige valget for dette datasettet.** LOOCV er likevel inkludert fordi det kan dukke opp konseptuelt (forklare forskjellen) eller fordi koden kan presenteres og du må forklare hvorfor K-fold brukes i stedet.

K-fold CV — primærvalg for $n=5000$

Kode: `Predicted-26Exam-SOLVED (cv_lasso)`

Spørsmål: Forklar funksjonen. Hvorfor K-fold fremfor LOOCV for dette datasettet? Hvorfor brukes treningsfoldets gjennomsnitt i linje `(*)`?

Nøkkelsvar — K-fold vs LOOCV: - LOOCV: n modeller per $\lambda \rightarrow$ umulig for $n=5000$ over et grid av λ -verdier - K-fold ($K=5$): 5 modeller per $\lambda \rightarrow$ 200 tilpasninger totalt $\rightarrow$ gjennomførbart - For lite n ($n=10$ i 2024, $n=100$ i 2025): LOOCV er fint; for $n=5000$: K-fold er nødvendig

****Nøkkelsvar — treningsgjennomsnittet i linje (*):**** `g()` demean X under trening — koeffisientene er kalibrert for demeaned input. Ved prediksjon: anvend treningsfoldets gjennomsnitt (ikke hele datasettets gjennomsnitt) — ellers lekker testinformasjon inn i feature-konstruksjonen og bryter hold-out-prinsippet.

LOOCV — kjenn mønsteret, forstå avveiningen

Dukket opp i 2024- og 2025-eksamen. Kode: `practice_exam_A (loo-funksjonen)`

Mønsteret: `mask[i] = False` $\rightarrow$ tren på $n-1 \rightarrow$ prediker obs i med treningsgjennomsnittet.

Spørsmål: Forklar funksjonen. Bruk den til å finne optimal parameter. Plot MSE (logg-skala). Er det nødvendig å demean $X_c[i]$ ved prediksjon? (Ja — koeffisientene er kalibrert for demeaned input.)

Kan dukke opp som: “Forklaringen under er en LOOCV-funksjon. Forklar hva den gjør og diskuter hvorfor K-fold CV er å foretrekke for et datasett av denne størrelsen.”

1(d) — Bootstrap (10 poeng)

Funksjonsstrukturen er alltid den samme. **Kun mål-statistikken varierer.**

Alle Bootstrap-varianter fra pensum:

Mål	Kode finnes i
Var(y) — stikkvariansen til responsen	<code>Predicted-26Exam-SOLVED (bootstrap_var)</code>
β_1 — en Lasso/Ridge-koeffisient	<code>practice_exam_A (bootstrap_coef)</code>
$\hat{f}(x_0)$ — prediksjon i fast punkt	<code>practice_exam_B (bootstrap_pred)</code>
Korrelasjon mellom to variabler	<code>BAN404_mock_exam-2</code>

Mål	Kode finnes i
Median av en variabel	BAN404_mock_exam-3
Porteføljevekt α (investeringseksempel)	Forelesning 6

Spørsmålene er alltid de samme — lær disse utenat: - Forklar hva funksjonen gjør og hvilken statistisk metode den implementerer. - Forklar linjene `(*)`, `(**)`, `(***)`. - Hvorfor **med tilbakelegging**? - Hvorfor samme `idx` for X og y? (Bevarer (X,y)-paret — uavhengig resampling ødelegger relasjonen.) - Rapporter bootstrap SE og 95% KI. Vis histogram med KI-grenser markert.

Nøkkelsvar — alltid gyldige:

`(*)` `rng.choice(n, size=n, replace=True)` → trekker n indekser med tilbakelegging. Kjennetegn på bootstrap: noen obs dukker opp flere ganger, andre ikke i det hele tatt.

`(**)` Konstruerer bootstrap-utvalget. Hvis paired (X,y): bruk *samme* idx for begge.

`(***)` Beregner målstatistikken på bootstrap-utvalget.

Hvorfor `replace=True`: Uten tilbakelegging returneres alltid originaldata (annen rekkefølge) → samme statistikk hver gang → ingen variasjon → ingen usikkerhetsmål.

95% KI: `np.percentile(boot, [2.5, 97.5])` — percentilbasert konfidensintervall.

1(e) — Utvidelse / ikke-linearitet (10 poeng)

Alle varianter fra pensum — ingen er trygt å utelukke:

GAM — Generalized Additive Model

Dukket opp i 2025-eksamen. Kode: `Predicted-26Exam-SOLVED`, `practice_exam_A`

```
from pygam import LinearGAM, s, l
# s(j) = smooth spline på prediktor j (ikke-lineær)
# l(j) = lineær term for prediktor j
gam = LinearGAM(l(0) + s(1) + s(2) + l(3) + l(4) + l(5))
gam.fit(X, y)

mse_null = np.mean((y - y.mean()) ** 2) # null-modell
mse_ols = np.mean((y - (X_aug @ b_ols)) ** 2)
mse_gam = np.mean((y - gam.predict(X)) ** 2)

r2_ols = 1 - mse_ols / mse_null
r2_gam = 1 - mse_gam / mse_null
```

Spørsmål: Plot y mot prediktorer. Identifiser ikke-lineære variabler (`s()`). Sammenlign trenings-MSE og R^2 med OLS. Hva forteller forbedringen om modellspesifikasjon?

Backfitting (konseptuelt — 2024-eksamen)

Ingen kode nødvendig. Forstå prinsippet:

Med to prediktorer x_1 og x_2 : (1) Tilpass $f(x_0, x_1, y, K) \rightarrow$ residualer $e_1 = y - \hat{y}_1$. (2) Tilpass $f(x_0, x_2, e_1, K) \rightarrow$ residualer e_2 . (3) Tilpass $f(x_0, x_1, e_2, K)$ igjen. Gjenta til konvergens $\rightarrow$ separate funksjoner $f(x_1)$ og $f(x_2)$.

Multivariat KNN (Euklidisk avstand)

Dukket opp i 2024-eksamen (1d) og `practice_exam_B` (1e). Kode: `practice_exam_B`

Nøkkelending: `d = np.sqrt(np.sum((X - x0)**2, axis=1))` i stedet for `np.abs(x - x0)`. Viktig: **standardiser prediktorer** før avstandsberegning — ellers dominerer variabler med stor skala og de med liten skala får nesten ingen innflytelse på hvem som velges som naboer.

Bagging og Random Forest (for regresjon)

Kode: `BAN404_mock_exam-2`

Spørsmål: Forklar koden. Sammenlign trenings- og test-MSE med OLS. Beregn R^2 for begge modeller på testsettet.

LDA / QDA vs. Logistisk regresjon

Kode: `BAN404_mock_exam-3`

LDA antar: $P(X|Y=k) \sim N(\mu_k, \Sigma)$ med felles kovariansmatrise $\rightarrow$ lineær beslutningsgrense. QDA antar: $P(X|Y=k) \sim N(\mu_k, \Sigma_k)$ med ulike kovariansmatriser $\rightarrow$ kvadratisk grense. Logistisk regresjon: ingen distribusjonsantagelse, modellerer $P(Y=1|X)$ direkte.

Spørsmål: Under hvilke forutsetninger er LDA/QDA bedre enn logistisk regresjon? (Svar: når normalitetsantagelsene holder — LDA/QDA bruker strukturen mer effektivt med lite data.)

PCA

Kode: se Python-kode i 1(a)-seksjonen over. `BAN404_mock_exam-3`

Spørsmål: Lag scree-plot. Hvor mange komponenter trengs for X% forklart varians? Tolk loadings for PC1 — hvilke variabler bidrar mest?

TASK 2 — Anvendt analyse (50 poeng)

Datsett: `customer_data.csv` — 2666 kunder, binær `Churn` (14.6% positive).

Analyserammen er stabil: Dataforberedelse $\rightarrow$ EDA $\rightarrow$ LR $\rightarrow$ RF $\rightarrow$ GB $\rightarrow$ Sammenligning. Men **delspørsmålene kan variere** — ikke bare svar på det "forventede".

2(a) — Dataforberedelse

Mulige spørsmål: - Kode om kategoriske variabler. Hvilke, og hvordan? - Identifiser perfekt kollineære par. Droppvalg og begrunnelse. - Train/test-split. Stratifisering — hvorfor? - Evaluerer du på trenings- eller testdata

for "hvorfor churn"? (→ Testdata: generalisering) - Er utfallet klassebalansert? Implikasjoner for valg av evalueringsmetrikk?

Nøkkelvalg for `customer_data.csv`: Se Predicted-26Exam-SOLVED (alle drop og encodings). Dropp: Total day/eve/night/intl charge (korr=1.0 med minutter), State (51 nivåer), Area code .

Klasseubalanse (mulig tilleggsspørsmål)

```
# class_weight='balanced' vektet minoritetsklassen høyere automatisk
lr_bal = LogisticRegression(class_weight='balanced', max_iter=2000)
rf_bal = RandomForestClassifier(n_estimators=500, class_weight='balanced')
# 'balanced' → vekt for klasse k = n / (n_classes * n_k)
# Effekt: modellen er mer oppmerksom på churners → høyere sensitivity, lavere specificity
```

2(b) — EDA

Mulige spørsmål: Deskriptiv statistikk + plot for å finne variabler assosiert med Churn. Hvilke tre variabler er sannsynligvis sterkest, og hvorfor?

Kode: Se Predicted-26Exam-SOLVED .

Nøkkelfunn for `customer_data.csv`:

Variabel	Churners	Ikke-churners
International plan = Yes	43.7% churn	11.3% churn
Customer service calls ≥ 4	48–100% churn	~10% churn
Total day minutes (gjsn.)	205 min	175 min
Voice mail plan = Yes	8.9% churn	16.7% churn

2(c) — Logistisk regresjon

Mulige spørsmål: - Accuracy, forvirringsmatrise, sensitivity, specificity, ROC, AUC. - Er terskel 0.5 passende? (Nei — imbalanced data, falske negativer er kostbare.) - Tolk koeffisientene for de to mest innflytelsesrike prediktorene. - Hva er implikasjonen av høy specificity men lav sensitivity?

Terskelanalyse (mulig tilleggsspørsmål)

```
for thresh in [0.2, 0.3, 0.4, 0.5, 0.6]:
    y_pred_t = (y_prob_lr >= thresh).astype(int)
    tn_t, fp_t, fn_t, tp_t = confusion_matrix(yte, y_pred_t).ravel()
    print(f"Terskel {thresh}: Sens={tp_t/(tp_t+fn_t):.3f}, Spec={tn_t/(tn_t+fp_t):.3f}")
# Lavere terskel → fanger flere churners (sensitivity↑) men mer falske alarmer (specificity↓)
```

Faktiske resultater: Accuracy=0.837, AUC=0.758, Sensitivity=0.167, Specificity=0.952

2(d) — Random Forest

Mulige spørsmål: - 5-fold CV på treningsdata for `max_features ∈ [2, 4, "sqrt", None]` . - Variabelviktigheter — plot og tolkning. Topp 5. Stemmer med EDA? - Sammenlign test-AUC med logistisk regresjon.

Kode: Se `Predicted-26Exam-SOLVED` .

Faktiske resultater: Best: `max_features=None` | AUC=0.878 | Sensitivity=0.718

2(e) — Gradient Boosting

Mulige spørsmål: - `staged_predict_proba` — plot test-AUC som funksjon av antall trær. - Hva er optimal B? Hva skjer med AUC etter det? - Refit med optimal B. Rapporter accuracy og AUC.

Kode: Se `Predicted-26Exam-SOLVED` .

Hva skjer etter optimal B: Test-AUC synker — overfitting. Hvert ekstra tre memorerer støy uten å forbedre generalisering. Illustrerer verdien av tidlig stopp.

Faktiske resultater: Optimal `B=113` | AUC=0.883 | Sensitivity=0.679

2(f) — Sammenligning og anbefaling

Modell	Accuracy	AUC	Sensitivity	Specificity
Logistisk Regresjon	0.837	0.758	0.167	0.952
Random Forest (None)	0.955	0.878	0.718	0.996
Gradient Boosting (B=113)	0.946	0.883	0.679	0.991

Anbefaling: GB eller RF — begge langt bedre enn LR på sensitivity (viktigst når falske negative er kostbare). GB har marginal AUC-fordel. Hvis fortolkbarhet er viktig: variabel- viktighetsplot fra RF gir tilstrekkelig innsikt for forretningsteamet.

Hurtigreferanse: Kode → Metode

Kodesignatur	Metode
<code>np.sum(np.abs(b))</code> i straff	Lasso
<code>np.sum(b ** 2)</code> i straff	Ridge
<code>np.abs(x - x0) + argsort()[:K]</code>	KNN lokal regresjon (1D)
<code>np.sqrt(np.sum((X - x0)**2, axis=1))</code>	KNN multivariat (Euklidisk)

Kodesignatur	Metode
<code>mask[i] = False</code> i løkke	LOOCV
<code>np.array_split(idx, K)</code>	K-fold CV
<code>rng.choice(n, replace=True)</code>	Bootstrap
<code>LinearGAM(s(...) + l(...))</code>	GAM
<code>PolynomialFeatures(degree=d)</code>	Polynomial regresjon
<code>pca.explained_variance_ratio_</code>	PCA
<code>staged_predict_proba</code>	GB med tidlig stopp
<code>feature_importances_</code>	RF / GB variabelviktighet
<code>LinearDiscriminantAnalysis()</code>	LDA
<code>QuadraticDiscriminantAnalysis()</code>	QDA

Nøkkelformler

```

R²          = 1 - MSE_model / MSE_null
MSE_null    = mean((y - y.mean())**2)      # null-modell = bare bruk mean(y)
Sensitivity  = TP / (TP + FN)              # andel churners korrekt identifisert
Specificity  = TN / (TN + FP)              # andel ikke-churners korrekt
identifisert
Bootstrap 95% = np.percentile(boot, [2.5, 97.5])
AUC         = areal under ROC-kurven (1.0 = perfekt, 0.5 = tilfeldig)

```

Basert på: BAN404_exam_2024 + 2025 (inkl. sensorveiledninger), forelesninger 2–14, practice_exam_A/B, BAN404_mock_exam-1/2/3, Predicted-26Exam-SOLVED, TestExam1.